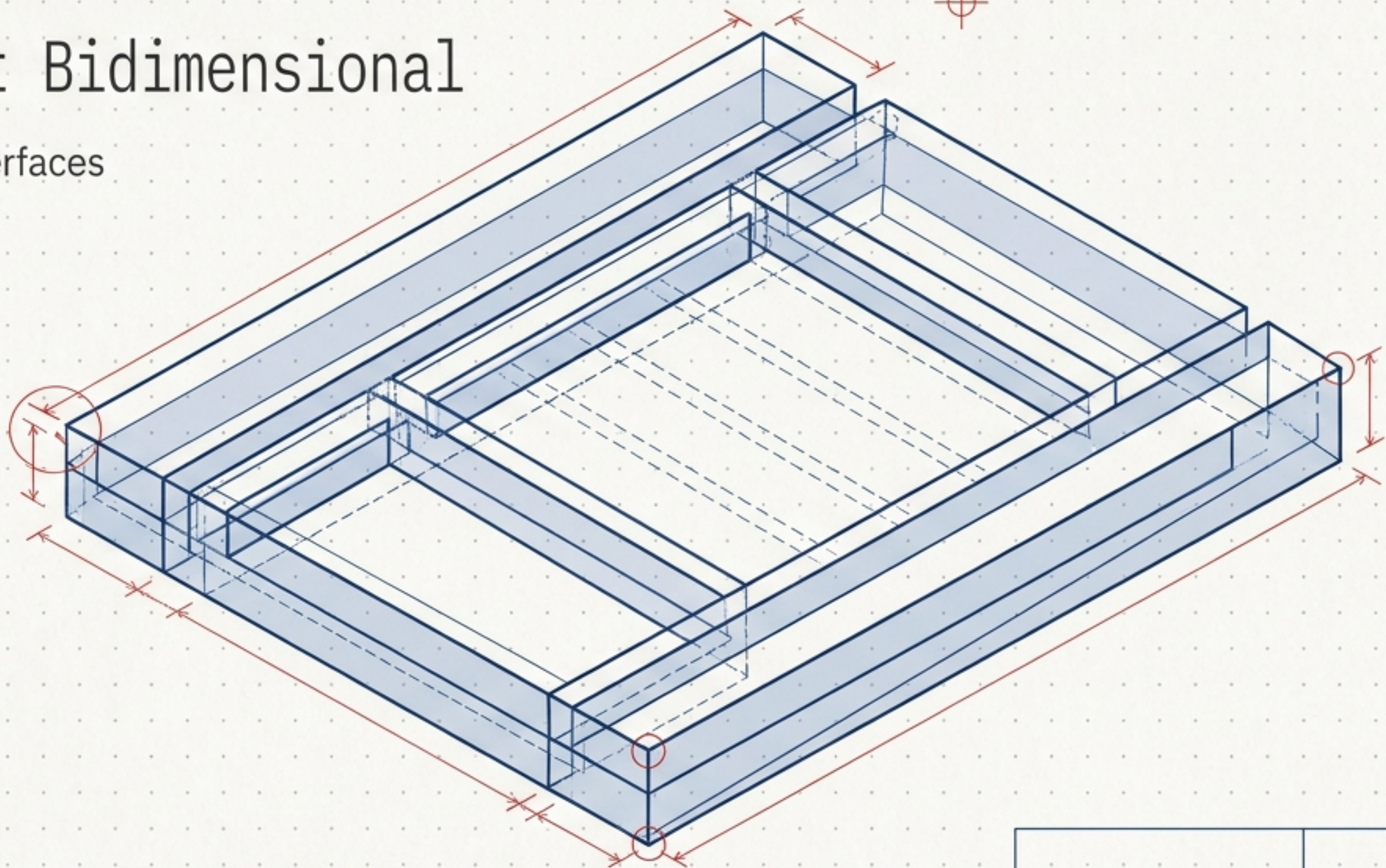


Dominando o CSS Grid

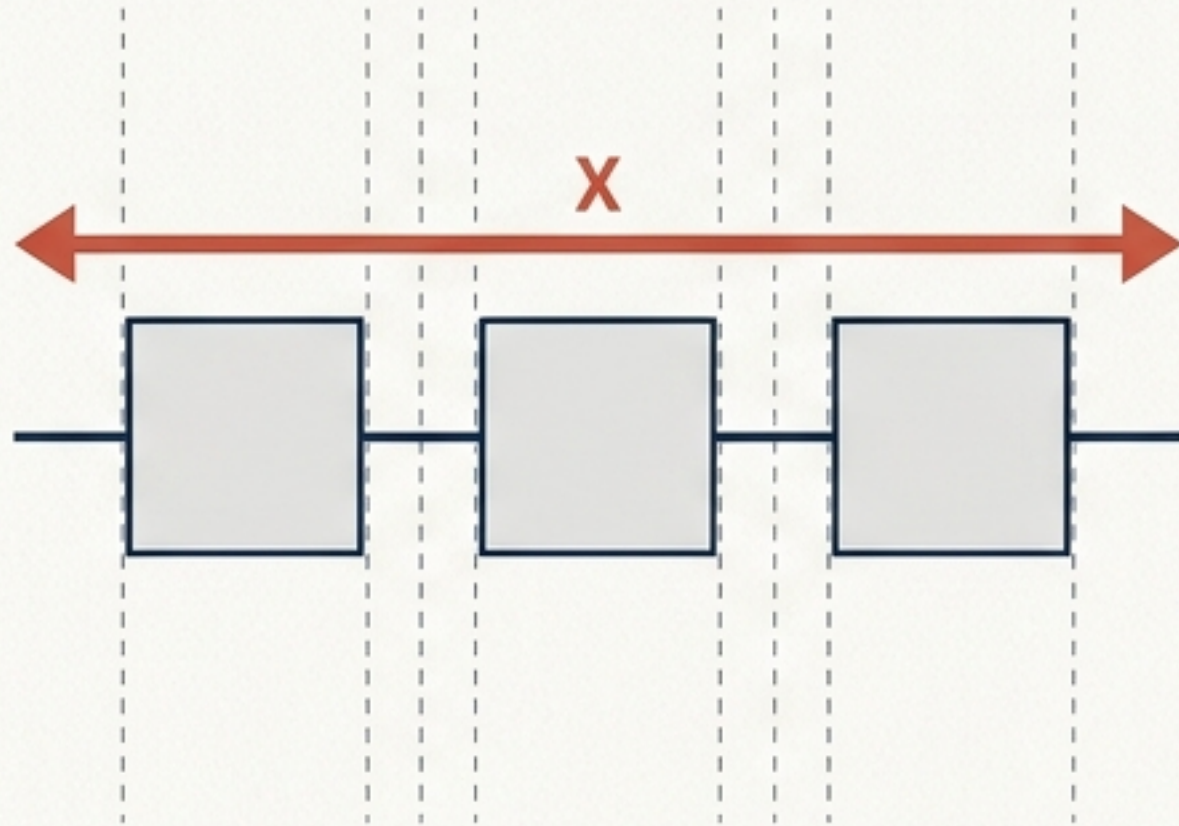
A Arte do Layout Bidimensional

Da teoria à estruturação de interfaces complexas sem media queries.



O Duelo de Layouts: Grid vs. Flexbox

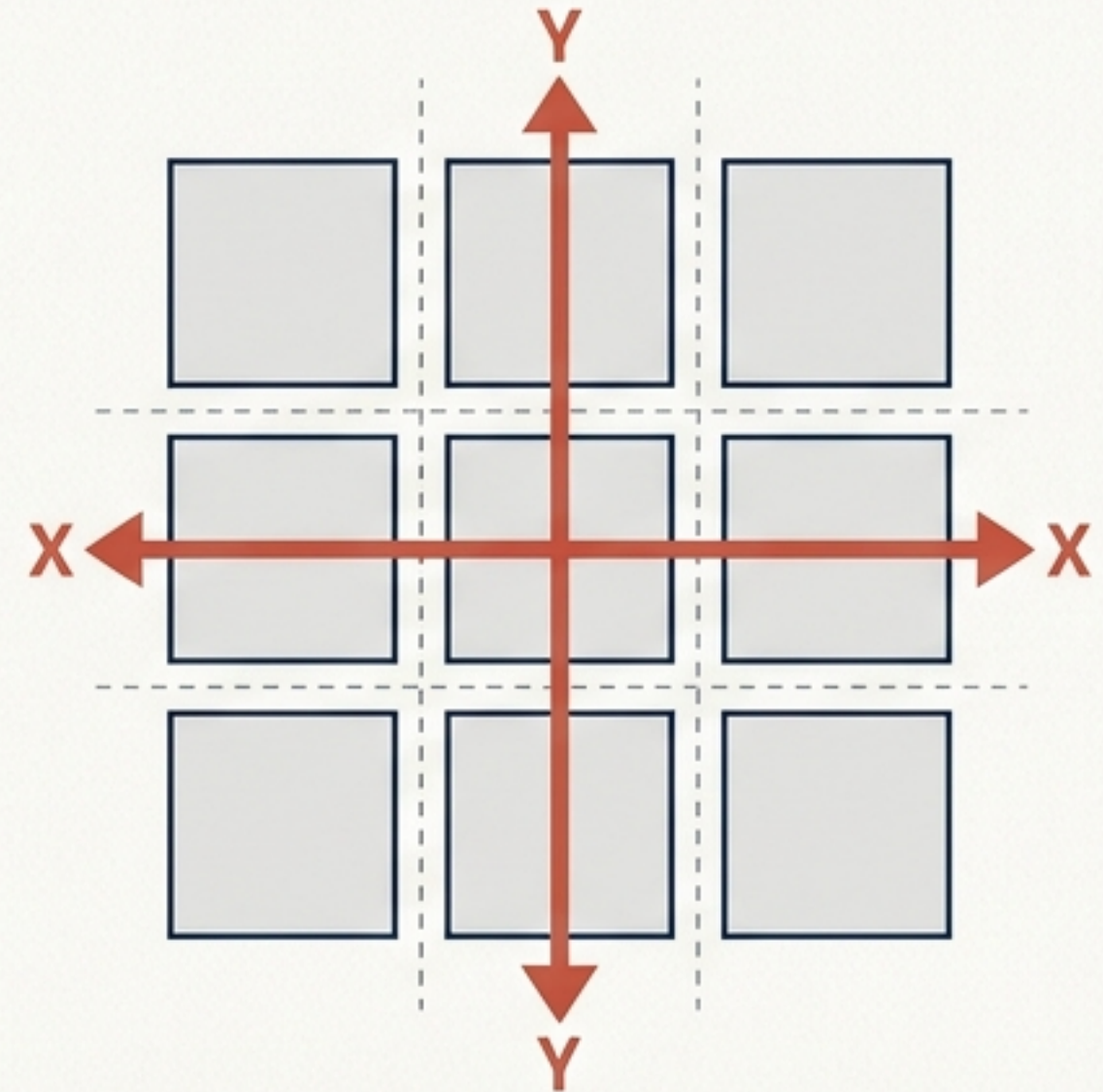
Flexbox



Foco: Unidimensional (linhas ou colunas)

Caso de Uso: Ideal para componentes internos e alinhamentos pontuais.

CSS Grid

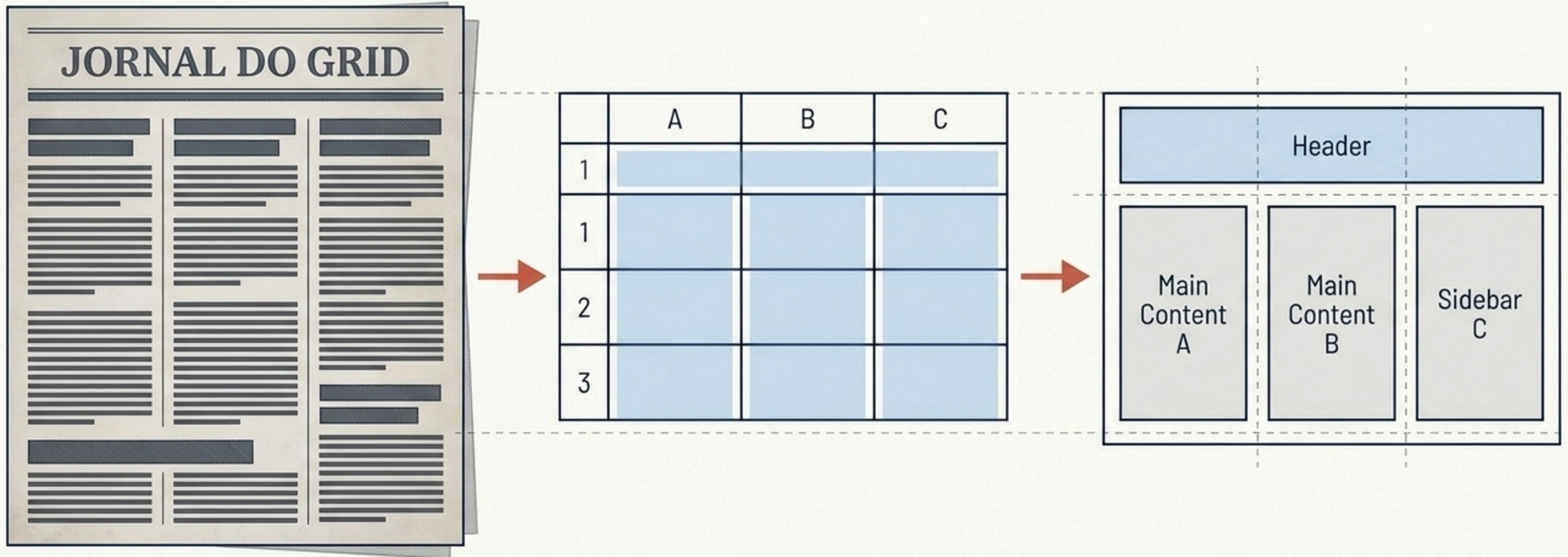


Foco: Bidimensional (linhas e colunas simultaneamente)

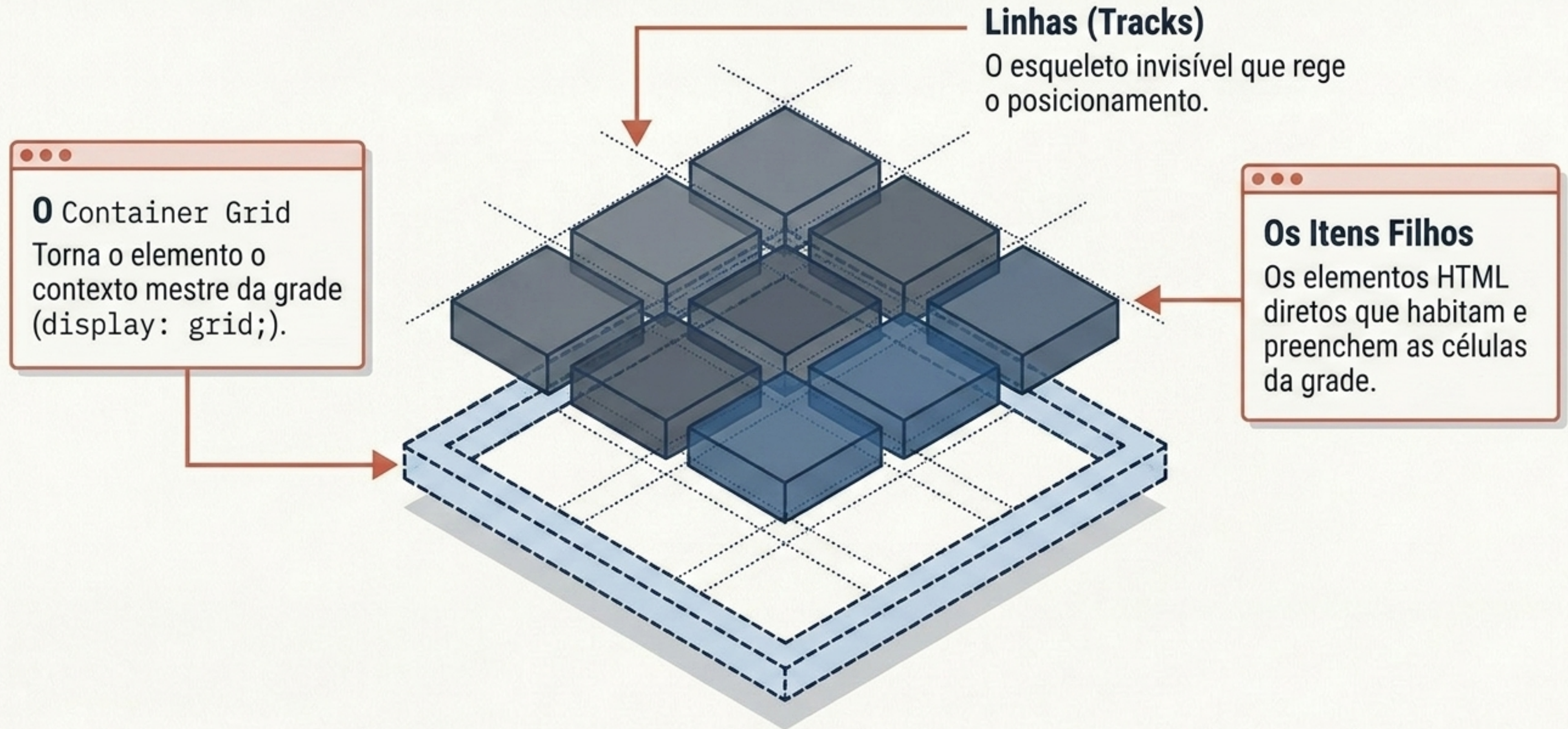
Caso de Uso: Ideal para o layout macro da página e estruturas complexas.

O Modelo Mental do Grid

Pense em uma folha de jornal ou planilha do Excel. Você não posiciona os elementos livremente; você primeiro desenha as **faixas (linhas e colunas)** e, em seguida, **encaixa** o conteúdo exatamente nos espaços correspondentes.

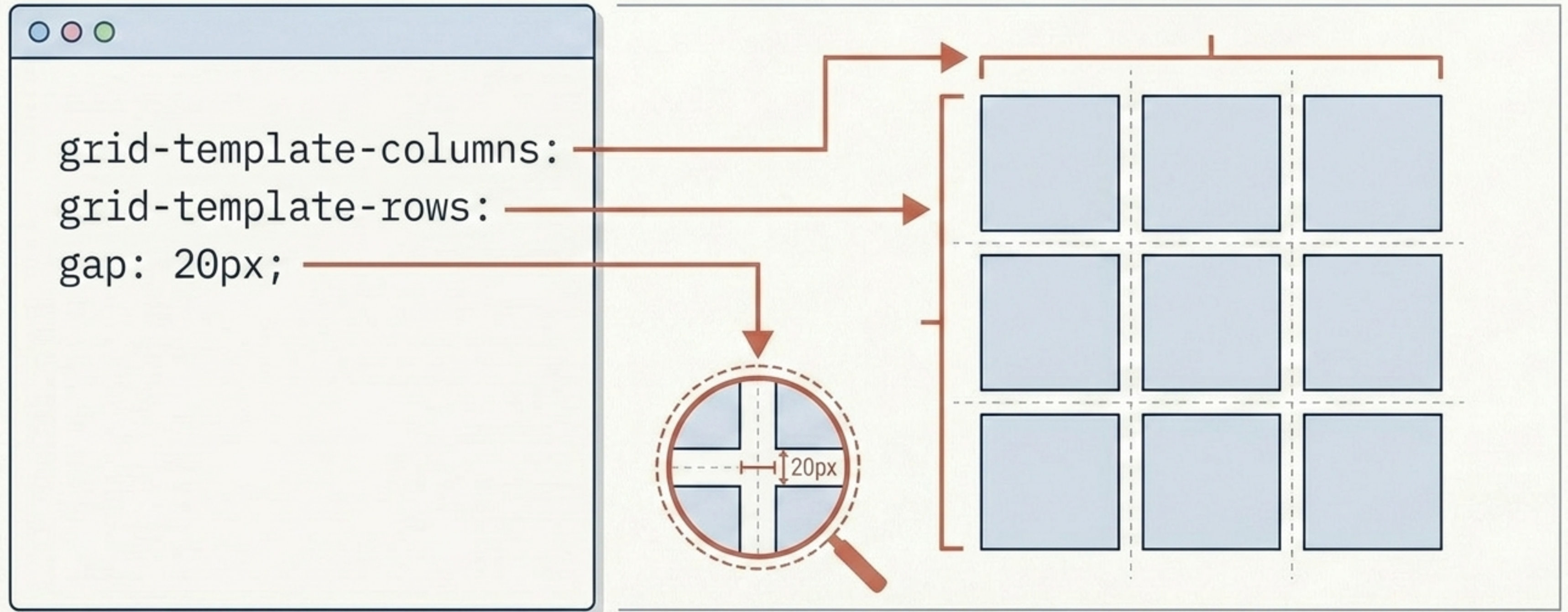


Dissecando a Anatomia do Grid



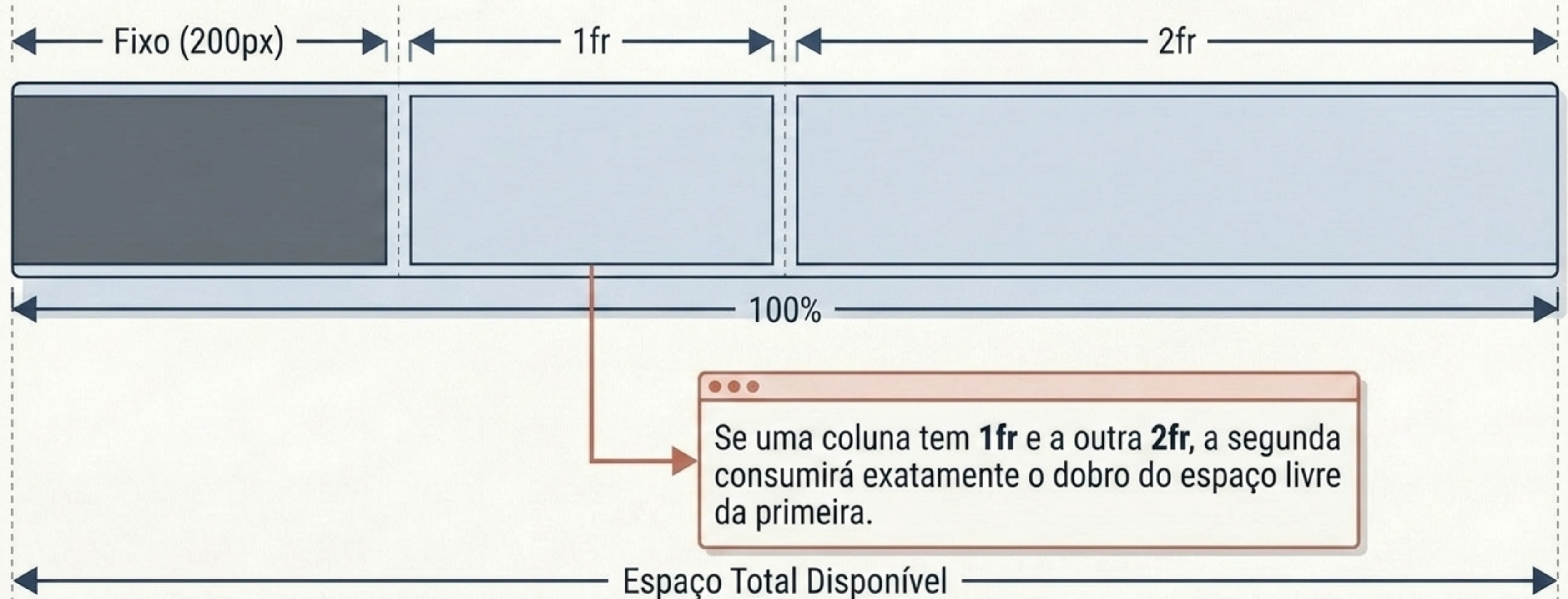
Definindo o Esqueleto

Colunas e Linhas definem o tamanho das faixas. O Espaçamento cria o respiro automático entre as células sem margens.



O Coringa do Grid: A Unidade Fracionária (fr)

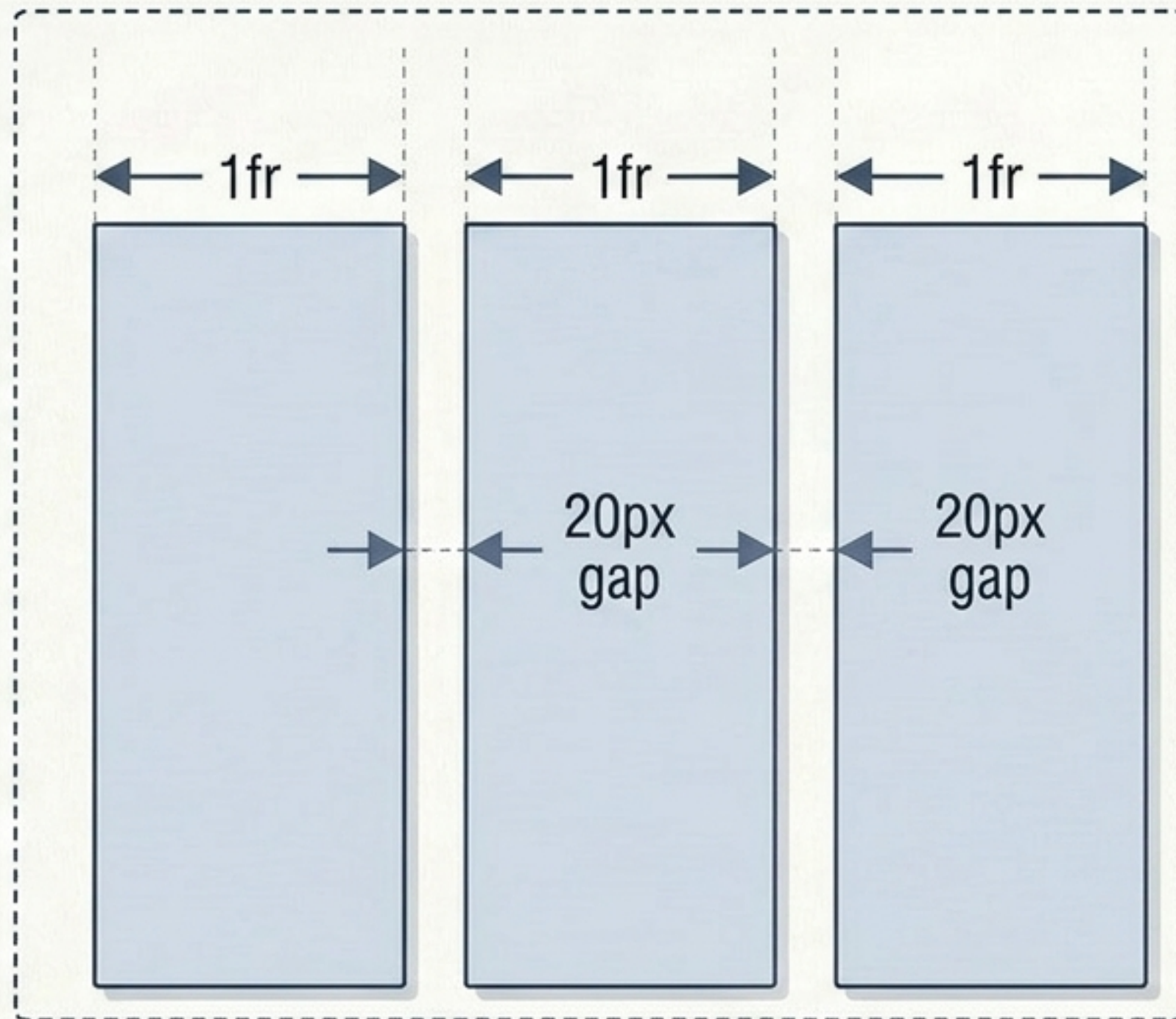
A unidade fr divide o espaço **restante** disponível proporcionalmente.



Exemplo Prático 1: O Grid Básico (3 Colunas)

```
.grid-container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 20px;  
}
```

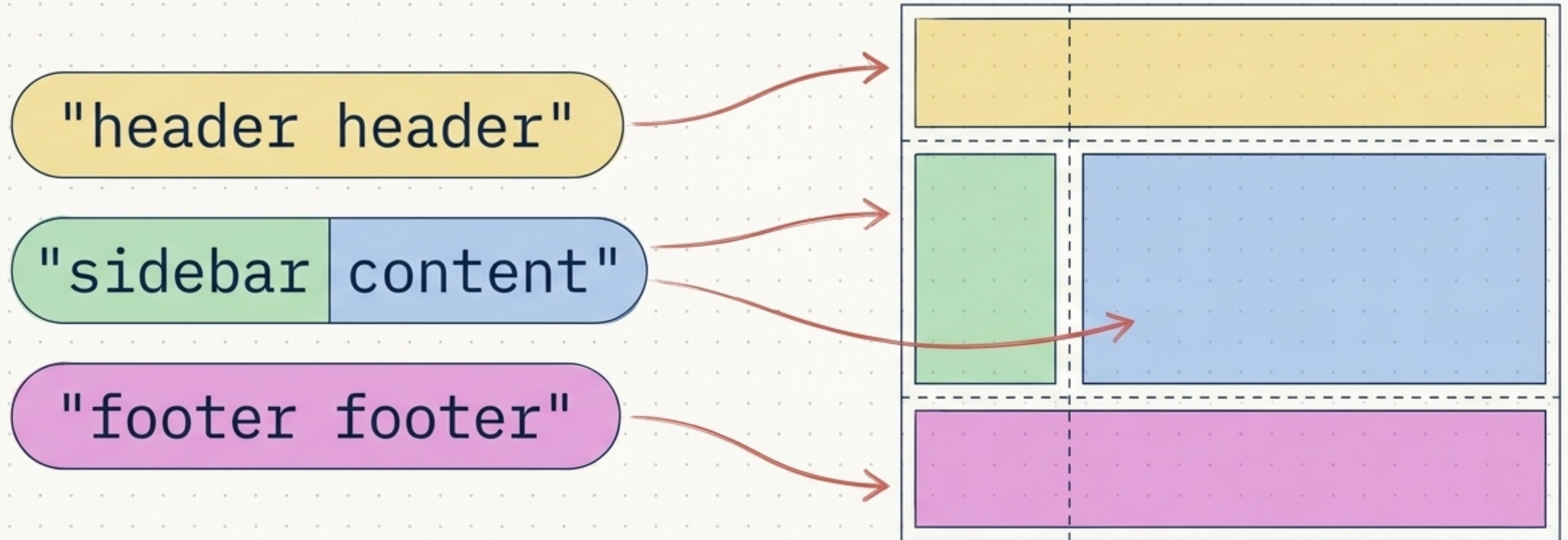
Função utilitária que
substitui escrever
1fr 1fr 1fr.



Resultado: 3 colunas fluidas perfeitamente iguais, com espaçamento constante.

O Poder Visual das Áreas Nomeadas

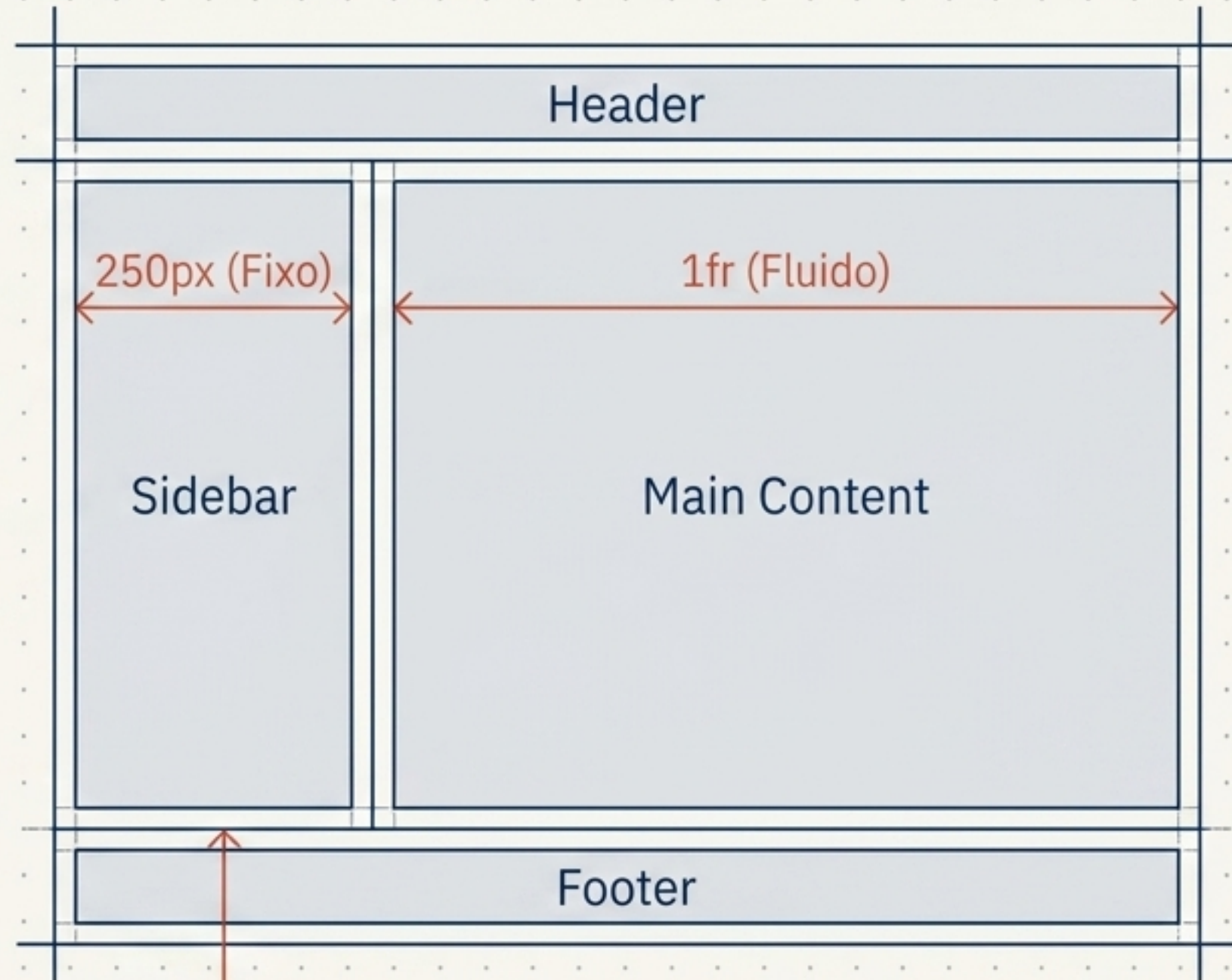
O grid-template-areas permite que você desenhe seu layout usando texto.
O CSS se torna uma representação visual direta da estrutura final.



Exemplo Prático 2: Layout Clássico de Site

```
.site-layout {  
  display: grid;  
  grid-template-areas:  
    "header header"  
    "sidebar content"  
    "footer footer";  
  grid-template-columns: 250px 1fr;  
}
```

```
header { grid-area: header; }
```

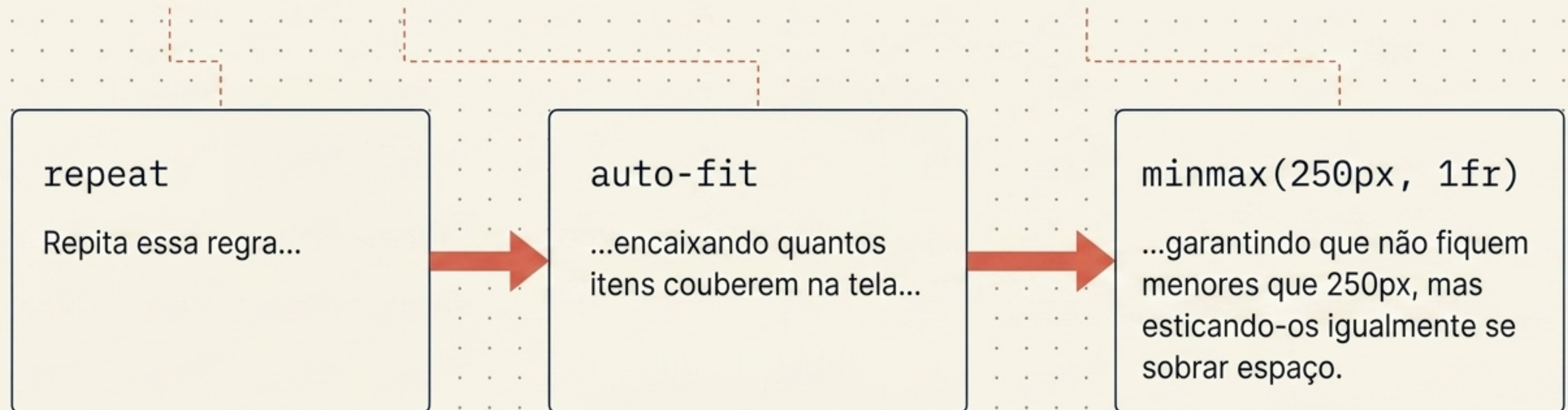


Note como 250px trava a sidebar, enquanto 1fr permite que o conteúdo respire e ocupe o restante do espaço.

O Segredo da Responsividade Orgânica

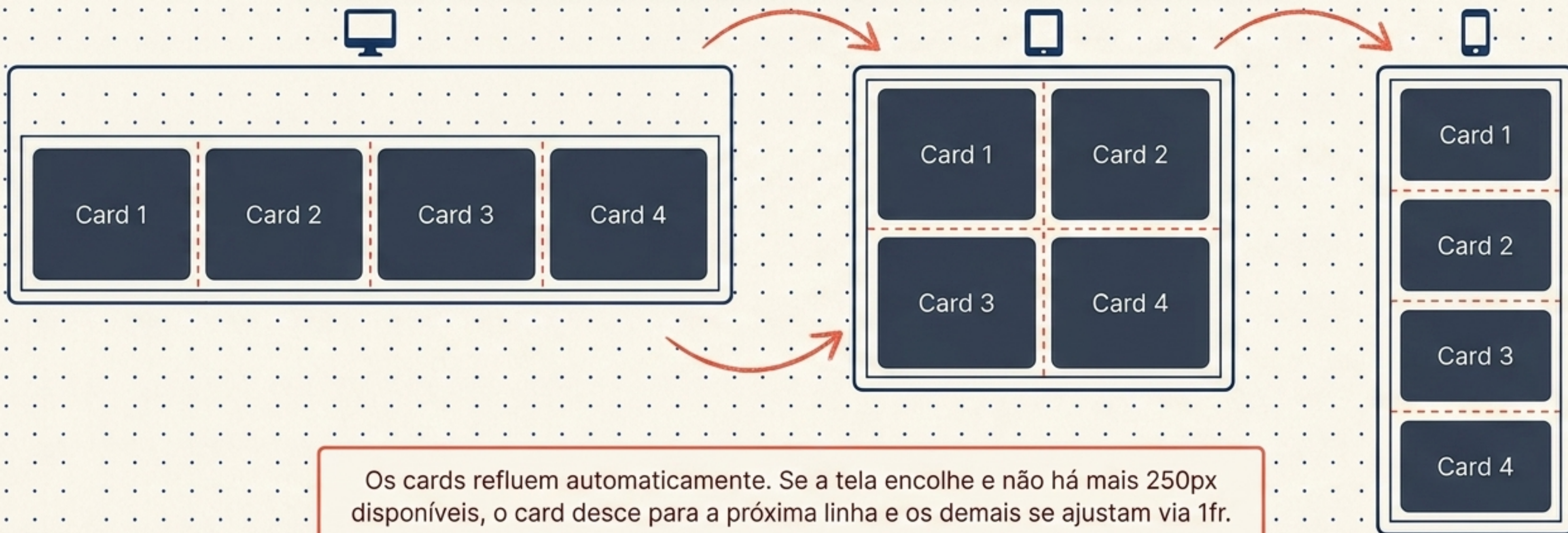
O verdadeiro poder **moderno do Grid** é criar layouts que se adaptam perfeitamente a qualquer tela sem escrever uma única Media Query.

```
repeat(auto-fit, minmax([tamanho-mínimo], 1fr))
```



Exemplo Prático 3: A Galeria Mágica

```
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
```

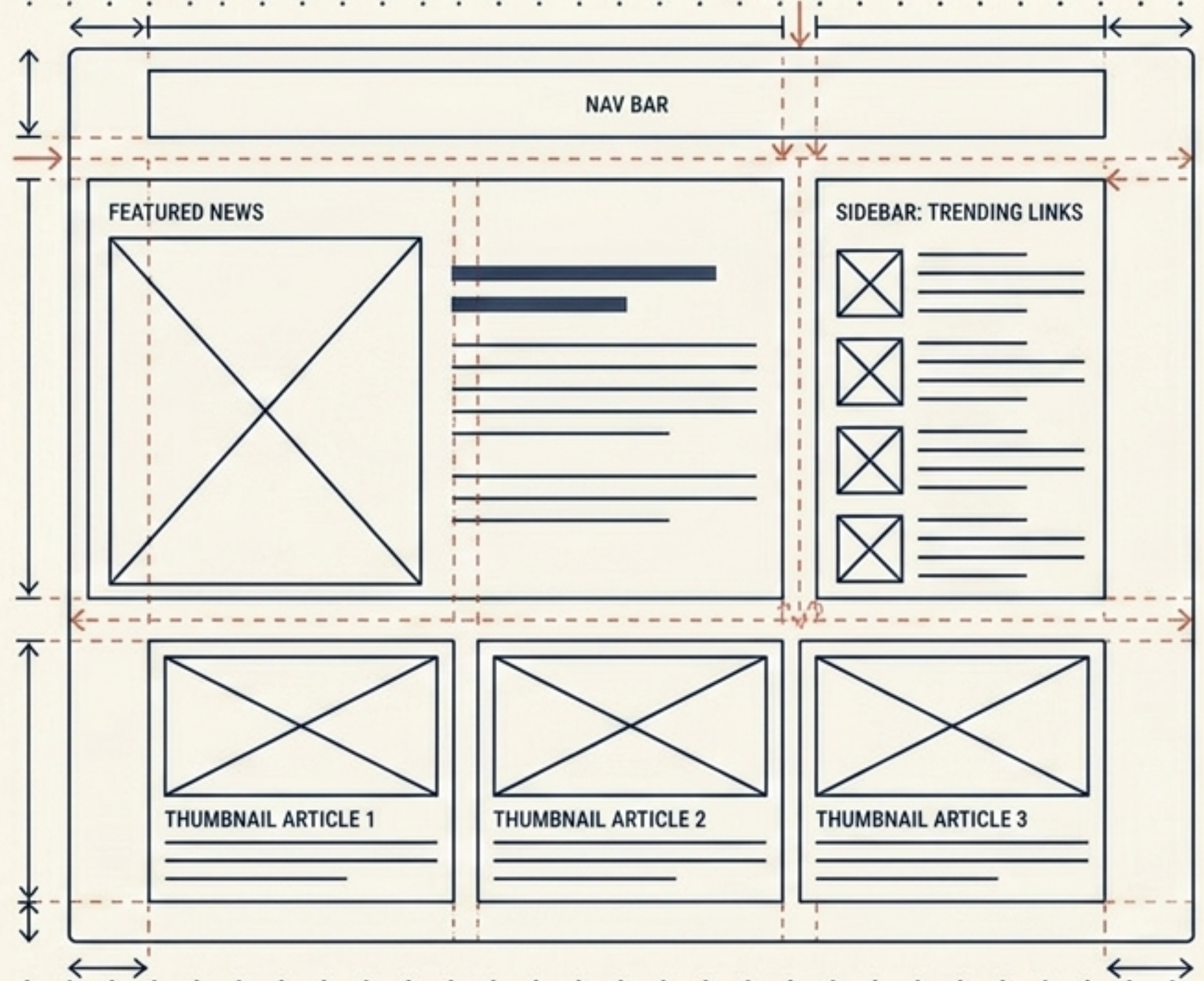


Desafio Prático: Construindo o Portal

Missão: Transformar este design estático em uma página responsiva.

Regras:

- Utilizar exclusivamente CSS Grid para a estrutura macro.
- Proibido o uso de **Flexbox** para o esqueleto principal.
- Planejem as columns, rows e areas no papel antes de codificar.



O Framework de 4 Passos do Mestre

1	☒ 0 Contexto <> <pre>... display: grid;</pre>	Ativou a grade no container pai?
2	☒ 0 Esqueleto <> <pre>... grid-template-columns / rows</pre>	Definiu as faixas usando a unidade fr?
3	☒ 0 Respiro <> <pre>... gap</pre>	Garantiu o espaçamento interno limpo sem usar margens?
4	☒ A Adaptação <> <pre>... repeat() + minmax()</pre>	Ativou a responsividade mágica para reflow automático?

O Grid não é sobre decorar código; é sobre arquitetar o espaço.
Construa sua planta baixa primeiro.